
On the Surprising Effectiveness of Fixed Feedback Weights for Language Model Supervised Fine-Tuning

Abstract

We study whether feedback alignment (FA) can be used for supervised fine-tuning of large language models. At linear operations, FA replaces the transposes of current forward weights used by backpropagation (BP) with fixed feedback matrices. We compare BP with two FA variants: FA-pretrained, which fixes the feedback connectors to the transposes of the pretrained weights, and FA-random, which uses fixed random connectors. Across full-parameter fine-tuning of Qwen2.5-3B and Qwen2.5-7B on GSM8K and SAMSum, FA-pretrained closely matches BP, while FA-random is generally worse but improves over the base model in every setting. Our endpoint analysis shows that FA-pretrained has nearly the same relative parameter drift as BP and remains strongly aligned with the BP endpoint update, whereas FA-random moves approximately twice as far in a nearly orthogonal direction. These results are consistent with the hypothesis that limited parameter drift during supervised fine-tuning allows pretrained weights to remain useful fixed feedback connectors.

1. Introduction

Backpropagation (BP), which computes parameter gradients by propagating loss derivatives backward through the computation graph, is the standard algorithm for training neural networks (Rumelhart et al., 1986). At a linear operation, this computation uses the transpose of the current forward weight matrix. In particular, the backward pathway must have access to and remain synchronized with the forward weights as they change during training. This requirement is known as the weight-transport problem and is difficult to satisfy in systems with separate forward and backward pathways, such as biological neural circuits (Grossberg, 1987; Crick, 1989). Feedback alignment (FA), introduced by Lillincrap et al. (2016), addresses the weight-transport problem by propagating backward error signals at each linear operation using a fixed random feedback matrix. The forward computation and the local derivatives of nonlinear opera-

tions remain unchanged.¹ More recently, Cheon et al. (2025) showed that imposing a one-time alignment between the forward weights and the fixed backward weights substantially improves FA in multilayer perceptrons and convolutional networks.

In this note, we apply FA to multi-billion-parameter language models. We focus on supervised fine-tuning (SFT), which adapts a pretrained model using prompt–response demonstrations by minimizing the cross-entropy loss on response tokens (Ouyang et al., 2022; Touvron et al., 2023). Unlike pretraining from random initialization, SFT begins from a pretrained base model and specializes it for downstream tasks. Because SFT adapts an already pretrained model, we hypothesize that its parameters remain closer to their initial values than they would during pretraining, allowing fixed feedback weights to remain useful throughout training. Our parameter-drift analysis confirms that the fine-tuned weights remain close to their pretrained values in our experiments (Section 3.3). This also makes the transposes of the pretrained weights a natural choice of fixed feedback connectors: at the beginning of SFT, they are exactly the connectors that BP would use, and they may remain a useful approximation as the weights change. We compare these fixed pretrained feedback connectors with fixed random connectors.

We first review the BP update for a linear module. Consider any linear module m inside a neural network computation graph. At optimization step s , its local forward computation is

$$z_m = W_{m,s} h_m. \quad (1)$$

For any graph variable v , let δ_v denote the backward error signal at that variable. In particular, δ_{z_m} is the signal arriving at the module output from the rest of the graph, and δ_{h_m} is the signal that the module transmits to its input. Under BP, these signals are the exact loss derivatives:

$$\delta_{z_m} = \frac{\partial \mathcal{L}}{\partial z_m}, \quad \delta_{h_m} = \frac{\partial \mathcal{L}}{\partial h_m} = W_{m,s}^\top \delta_{z_m}, \quad (2)$$

¹A related method, direct feedback alignment (DFA), projects the output error directly to intermediate modules through fixed feedback matrices, rather than propagating it sequentially backward through the computation graph (Nøkland, 2016). We return to DFA as a direction for future work in Section 4.

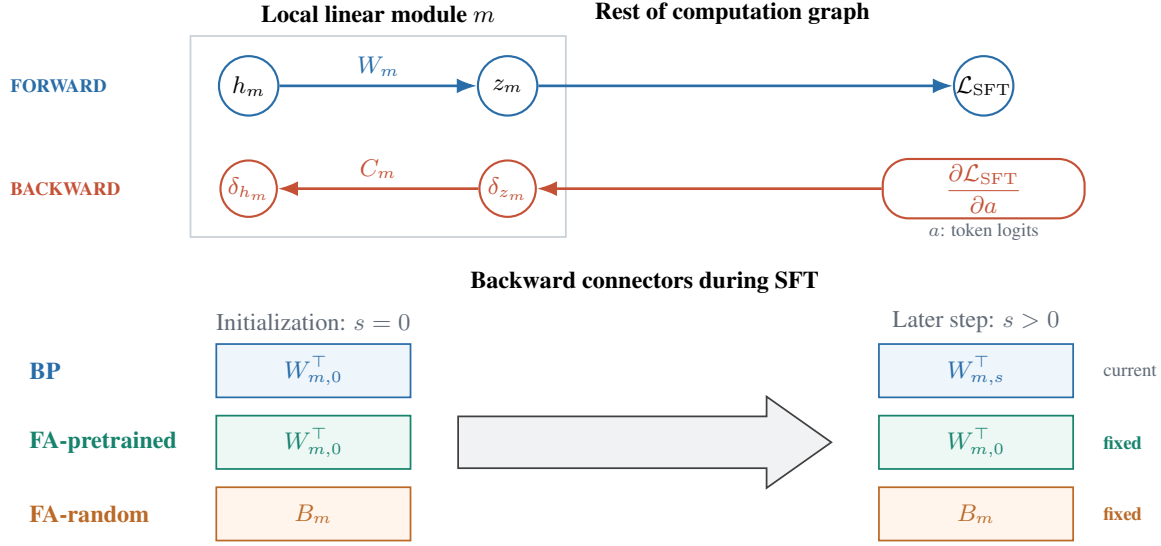


Figure 1. In FA, backward signals are recursively propagated through the computation graph using fixed backward connectors at selected linear modules. During SFT, the BP backward connector changes with the forward weight, whereas the FA-pretrained and FA-random connectors remain fixed.

and the weight update is

$$\Delta W_{m,s} = -\eta \frac{\partial \mathcal{L}}{\partial W_{m,s}} = -\eta \delta_{z_m} h_m^\top. \quad (3)$$

Here η is the learning rate. Although $W_{m,s}^\top$ does not appear in the local update of $W_{m,s}$, it is required to send the error signal δ_{h_m} to the operations preceding module m , where that signal determines upstream parameter updates. FA changes this transmitted signal.

Example 1. Consider a multilayer neural network. At layer l , the local forward computation is

$$o_{l+1} = W_l h_l, \quad h_{l+1} = \phi(o_{l+1}), \quad (4)$$

where ϕ is a nonlinear activation function acting coordinate-wise. Under BP, the backward signals through the nonlinear and linear operations are

$$\delta_{o_{l+1}} = \delta_{h_{l+1}} \odot \phi'(o_{l+1}), \quad \delta_{h_l} = W_l^\top \delta_{o_{l+1}}, \quad (5)$$

and the corresponding weight update is

$$\Delta W_l = -\eta \frac{\partial \mathcal{L}}{\partial W_l} = -\eta \delta_{o_{l+1}} h_l^\top. \quad (6)$$

Feedback alignment (FA) replaces the BP connector in (2) with

$$\delta_{h_m} = B_m \delta_{z_m}, \quad (7)$$

where B_m is a fixed feedback matrix. Globally, FA propagates signals backward through the computation graph, using the custom rule in Equation (7) at selected linear modules (Section 2) and the standard BP backward rules at all

other operations. Consequently, the propagated δ_v signals need not equal the corresponding loss derivatives. In the original FA paper, Lillicrap et al. (2016) proposed choosing B_m randomly; we call this variant FA-random. We also consider setting $B_m = W_{m,0}^\top$, the transpose of the pretrained forward weight matrix. We call this FA-pretrained. FA-pretrained therefore agrees exactly with BP at the beginning of SFT. Afterward, BP uses the current connector $W_{m,s}^\top$, whereas FA-pretrained continues to use the fixed connector $W_{m,0}^\top$.

We perform full-parameter SFT under BP, FA-pretrained, and FA-random, applying the FA replacement to selected attention and MLP linear projections (see Section 2). Across two model sizes and two tasks, FA-pretrained closely tracks BP, whereas FA-random is less consistent but still improves over the base model in every setting (Table 1). FA-random performs below BP in three settings but exceeds the observed BP mean for 7B GSM8K. Our parameter-drift and endpoint-update analyses show that FA-pretrained has nearly the same relative drift as BP and remains strongly aligned with the BP endpoint update (Tables 2 and 3). FA-random instead moves approximately twice as far and follows a nearly orthogonal update direction. These observations are consistent with the hypothesis that limited parameter drift allows the pretrained weights to remain useful fixed feedback connectors.

2. Method

All three training conditions use the same standard token-level cross-entropy objective. Given a prompt x and target

response tokens $y_1, \dots, y_k$, the SFT loss is

$$\mathcal{L}_\theta = - \sum_{t=1}^k \log P_\theta(y_t | x, y_1, \dots, y_{t-1}). \quad (8)$$

The three training conditions differ only in the error signal that each selected linear module sends to its input:

$$\delta_{h_m} = \begin{cases} W_{m,s}^\top \delta_{z_m}, & \text{BP,} \\ W_{m,0}^\top \delta_{z_m}, & \text{FA-pretrained,} \\ B_m \delta_{z_m}, & \text{FA-random,} \end{cases} \quad (9)$$

where $W_{m,0}$ is the module’s pretrained weight and B_m is a randomly initialized feedback matrix of the appropriate

shape. Under BP, $\delta_{z_m} = \partial \mathcal{L} / \partial z_m$ and $\delta_{h_m} = \partial \mathcal{L} / \partial h_m$; under either FA variant, the δ_v are the backward signals produced recursively by the corresponding feedback rule and are not generally exact loss derivatives. We apply Equation (9) to the four linear projections in each attention block—the query, key, value, and output projections—and to the three MLP projections: gate, up, and down. All other operations retain the standard local derivatives used by BP. These include linear and bilinear operations, such as the language-model head and the attention matrix multiplications $QK^\top$ and AV , as well as nonlinear operations, such as softmax and the SiLU activation in SwiGLU.

Table 1. Downstream task performance. Higher values are better for both metrics.

Model	Task (metric)	Base	BP	FA-pretrained	FA-random
3B	GSM8K (exact match)	0.004	0.665 ± 0.013	0.659 ± 0.008	0.637 ± 0.024
3B	SAMSum (ROUGE-L)	0.347	0.442 ± 0.001	0.440 ± 0.002	0.383 ± 0.006
7B	GSM8K (exact match)	0.080	0.752 ± 0.004	0.748 ± 0.009	0.763 ± 0.005
7B	SAMSum (ROUGE-L)	0.371	0.452 ± 0.005	0.450 ± 0.008	0.397 ± 0.004

3. Experiments

We compare the three methods on Qwen2.5-3B and Qwen2.5-7B using GSM8K and SAMSum.

3.1. Experimental Setup

Following [Gerontopoulos et al. \(2025\)](#), we evaluate mathematical reasoning on GSM8K ([Cobbe et al., 2021](#)) and abstractive dialogue summarization on SAMSum ([Gliwa et al., 2019](#)). We use Qwen2.5-3B and Qwen2.5-7B ([Yang et al., 2024](#)) as our pretrained models. GSM8K contains 7,473 training examples and 1,319 test examples, while our processed SAMSum data contain 14,719 training, 818 validation, and 819 test examples.

All training runs use full-parameter SFT with AdamW, a learning rate of 10^{-5} , no weight decay, an effective batch size of 8, and a maximum sequence length of 768. GSM8K models are trained for four epochs, with checkpoints retained at epochs 3 and 4. SAMSum models are trained for three epochs, with a checkpoint retained after each epoch. We use seeds 1234, 2345, and 3456. Aggregated results are reported as the mean $\pm$ sample standard deviation over three seeds.

We use greedy decoding for both tasks, generating at most 512 new tokens for GSM8K and 128 for SAMSum. GSM8K is evaluated on the full test set using strict final-answer exact match. For SAMSum, we compute bootstrap ROUGE-L with stemming on the full validation and

test sets ([Lin, 2004](#)). For GSM8K, we report the scheduled checkpoint with the highest test exact match. These scores therefore describe the best observed scheduled checkpoints rather than held-out estimates after independent model selection. For FA on SAMSum, we select the checkpoint with the highest validation ROUGE-L and report its corresponding test ROUGE-L. For BP on SAMSum, however, we report the scheduled checkpoint with the highest test ROUGE-L. Implementation and experiment configurations are available at <https://github.com/leon2k2k2k/llm-feedback-alignment>.

3.2. Results

Table 1 summarizes the quantitative results. FA-pretrained closely tracks BP across both tasks and model sizes; the absolute differences between their observed means range from 0.002 to 0.006. FA-random is less consistent and performs below BP on 3B GSM8K and on SAMSum at both model sizes. The sole exception is 7B GSM8K, where FA-random obtains a higher observed mean than both BP and FA-pretrained. Both FA variants improve over the corresponding base model in every setting, showing that fixed feedback still supports substantial downstream improvement.

3.3. Analysis

We analyze the endpoint updates using matched epoch-3 GSM8K checkpoints for BP, FA-pretrained, and FA-random

Table 2. Relative endpoint parameter drift from the pretrained initialization, $\|\theta_T - \theta_0\|_2 / \|\theta_0\|_2$, at matched epoch-3 GSM8K checkpoints.

Model	Method	Relative drift
3B	BP	0.004387 ± 0.000002
3B	FA-pretrained	0.004386 ± 0.000004
3B	FA-random	0.009214 ± 0.000081
7B	BP	0.004996 ± 0.000007
7B	FA-pretrained	0.004994 ± 0.000007
7B	FA-random	0.009570 ± 0.000126

Table 3. Cosine similarity between each FA endpoint update and its matched BP endpoint update at the corresponding epoch-3 GSM8K checkpoints.

Model	Method	Cosine similarity
3B	FA-pretrained	0.9713 ± 0.0003
3B	FA-random	0.0561 ± 0.0012
7B	FA-pretrained	0.8361 ± 0.0034
7B	FA-random	0.0437 ± 0.0004

for each of the three seeds. For each method m , we define its endpoint update and relative parameter drift as

$$\Delta\theta_m = \theta_{m,T} - \theta_0, \quad d_m = \frac{\|\Delta\theta_m\|_2}{\|\theta_0\|_2}, \quad (10)$$

where θ_0 denotes the pretrained initialization and T denotes the epoch-3 analysis checkpoint. As shown in Table 2, BP and FA-pretrained have nearly identical relative drift at both model sizes. In contrast, FA-random moves approximately 2.10 times as far as BP at 3B and 1.92 times as far at 7B.

To compare the directions of these endpoint updates, we compute their cosine similarity, defined as

$$\text{CosSim}(\Delta\theta_{\text{FA}}, \Delta\theta_{\text{BP}}) := \frac{\langle \Delta\theta_{\text{FA}}, \Delta\theta_{\text{BP}} \rangle}{\|\Delta\theta_{\text{FA}}\|_2 \|\Delta\theta_{\text{BP}}\|_2}. \quad (11)$$

Table 3 shows that FA-pretrained is strongly aligned with BP, with cosine similarities of 0.971 at 3B and 0.836 at 7B. FA-random has cosine similarities of only 0.056 and 0.044, respectively, and therefore follows a nearly orthogonal direction in parameter space.

These results are consistent with a small-drift explanation for the success of FA-pretrained. BP transports error through the current forward weights $W_{m,s}^\top$, whereas FA-pretrained continues to use the fixed pretrained weights $W_{m,0}^\top$. Since SFT keeps $W_{m,s}$ close to $W_{m,0}$, the fixed pretrained pathway remains a useful approximation to the pathway that BP would use. Consistent with this explanation, the FA-pretrained endpoint update has nearly the same magnitude as its matched BP update and remains strongly aligned with

it, while random feedback produces a larger and nearly orthogonal update.

4. Conclusion and Discussion

We find that pretrained feedback provides an effective fixed backward pathway for LLM supervised fine-tuning. In our experiments, FA-pretrained closely matches BP across both tasks and model sizes, while remaining strongly aligned with the BP updates. These observations are consistent with a small-drift explanation: because SFT leaves the current weights close to their pretrained values, the fixed pretrained weights remain a useful approximation to the backward pathway used by BP. FA-random is less consistent and follows a larger, nearly orthogonal trajectory. Our experiments are limited to two tasks, two Qwen2.5 model sizes, three seeds, and relatively short SFT runs; we do not establish a computational advantage over BP.

One natural direction for future work is direct feedback alignment (DFA), which routes the output error directly to intermediate modules. In its simplest form, the local signal is $\delta_{h_m} = B_m e$, where e is the output error and B_m is a fixed feedback matrix for module m (Nøkland, 2016). DFA therefore offers a structural opportunity for parallel or local updates. Scaling-law evidence indicates that DFA scales unfavorably relative to BP in decoder-only transformers (Filipovich et al., 2022). Related work has applied DFA to modern architectures and BP-pretrained networks (Launay et al., 2020; Lee & Lee, 2026).

5. The Use of Large Language Models

Large language models (LLMs) were used solely for language polishing and editorial refinement of this manuscript, including grammar correction and clarity improvements. All research content, methodology, analysis, and conclusions are the original work of the authors.

References

- Cheon, J., Bae, J., and Paik, S.-B. One-time soft alignment enables resilient learning without weight transport. *arXiv preprint arXiv:2505.20892*, 2025. URL <https://arxiv.org/abs/2505.20892>.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Crick, F. The recent excitement about neural networks. *Nature*, 337(6203):129–132, 1989.
- Filipovich, M. J., Cappelli, A., Hesslow, D., and Launay,

- J. Scaling laws beyond backpropagation. *arXiv preprint arXiv:2210.14593*, 2022.
- Gerontopoulos, A., Gidaris, S., and Komodakis, N. Multi-token prediction needs registers. In *Advances in Neural Information Processing Systems*, volume 38, pp. 67010–67035, 2025. doi: 10.52202/085713-2013.
- Gliwa, B., Mochol, I., Biesek, M., and Wawer, A. SAM-Sum corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, pp. 70–79. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-5409. URL <https://aclanthology.org/D19-5409/>.
- Grossberg, S. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1): 23–63, 1987.
- Launay, J., Poli, I., Boniface, F., and Krzakala, F. Direct feedback alignment scales to modern deep learning tasks and architectures. In *Advances in Neural Information Processing Systems*, 2020.
- Lee, Y. and Lee, S. Enabling fine-tuning of direct feedback alignment via feedback-weight matching. In *International Conference on Learning Representations*, 2026.
- Lillicrap, T. P., Cownden, D., Tweed, D. B., and Akerman, C. J. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7:13276, 2016. doi: 10.1038/ncomms13276. URL <https://www.nature.com/articles/ncomms13276>.
- Lin, C.-Y. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pp. 74–81. Association for Computational Linguistics, 2004. URL <https://aclanthology.org/W04-1013/>.
- Nøkland, A. Direct feedback alignment provides learning in deep neural networks. In *Advances in Neural Information Processing Systems*, 2016.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.